

2019 年 863 参考答案

一、选择题

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
B	C	C	A	A	C	D	D	A	A	D	C	C	D	B

二、判断题

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
√	×	×	√	×	×	×	×	×	√	×	√	×	√	×

三、填空题

- 方便操作
- 假溢出
- 逻辑结构
- 顺序
- 最小生成树
- $O(e \log e)$
- 队列
- $\log_2(n+1)-1$
- $n-i+1$
- $n-i$
- 题目有误
- 4

11、 $\lfloor n/2 \rfloor$ $n-1$

12、出度

四、简答题

1. 折半搜索的判定树

ASL 查找成功 = $\frac{1 \times 2 \times 2 + 4 \times 3 + 7 \times 4}{14} = \frac{45}{14}$

ASL 查找失败 = $\frac{1 \times 3 + 14 \times 4}{15} = \frac{59}{15}$

2. (1) 线性探测

地址	0	1	2	3	4	5	6	7	8	9	10	11
关键字	43	56	24	35	44		28	18	39		21	32

$H(21) = 10$
 $H(18) = 7$
 $H(28) = 6$
 $H(24) = 2$
 $H(32) = 10$
 $H(56) = 1$
 $H(43) = 10, H_1(43) = (10+1) \% 12 = 11, H_2(43) = (10+2) \% 12 = 0$
 $H(35) = 2, H_1(35) = 3$
 $H(39) = 6, H_1(39) = 7, H_2(39) = 8$
 $H(44) = 0, H_1(44) = 1, H_2(44) = 2, H_3(44) = 3, H_4(44) = 4$

冲突时 % 表长

(2) ASL 线性查找成功 = $\frac{4+2+1+3+2+3+5}{10} = 2$

ASL 线性查找失败 = $\frac{1 \times 6 + 3 \times 2 + 3 \times 1}{10} = \frac{9}{2}$

(3) typedef int ElemType;

```

struct mynode
{
    ElemType data;
    mynode *next;
}
    
```

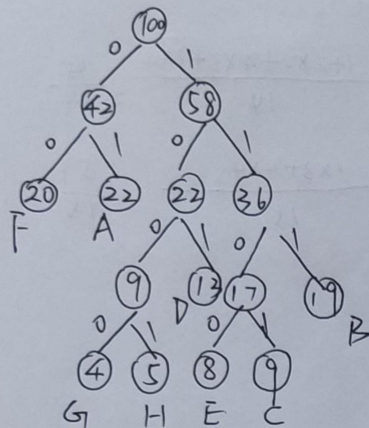
struct HashTab;

```

{
    mynode *data;
}
    
```

链地址法

3. 解: (1) 4 5 8 9 13 19 20 22



A: 01
B: 111
C: 1101
D: 101
E: 1100
F: 00
G: 1000
H: 1001

4. 邻接矩阵

	A	B	C	D	E	F
A	0	1	0	0	1	0
B	1	0	0	1	0	0
C	0	0	0	0	0	1
D	0	1	0	0	1	0
E	1	0	0	1	0	1
F	0	0	1	0	1	0

以 A 为例的
深: ABDEFC (不唯一)
广: ABEDFC

5. (1) 包含图中全部顶点的一个极小连通子图。

(2) 按边权从小到大排序, 每迭代一次就选择一条满足条件的最小代价的边, 并加入到最小生成树的边集中。

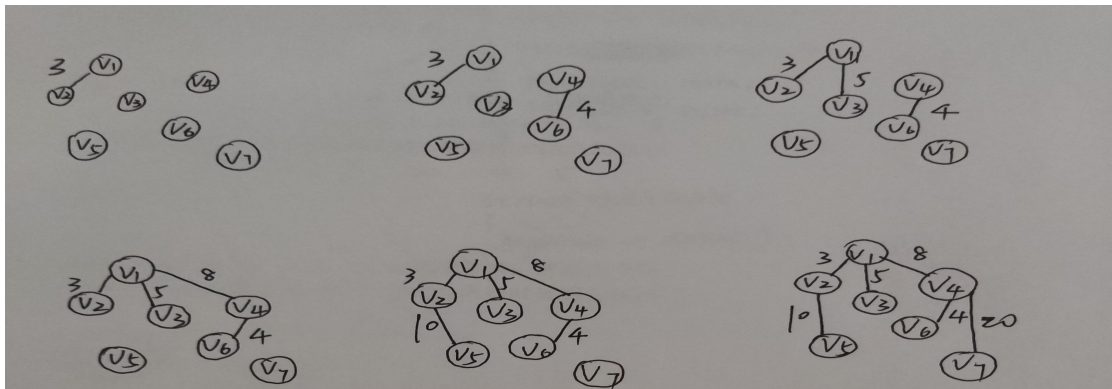
① 把图中的边按代价从小到大排序。

② 把图中 n 个顶点看成 n 棵独立的树组成的森林。

③ 按代价从小到大选取边, 所选的边连接的 2 个顶点 u, v 应属于两棵不同的树, 则成为最小生成树的一边, 并把这两棵树合并为一棵树。

④ 重复 ③, 直到最小生成树内有 $(n-1)$ 条边或 n 个顶点为止。

(3)



五、算法设计题

1019年

(1) 在链表中删除一个数据为x的结点

```
void Delete (LNode *head, int x)
```

```
{  
    LNode *p=head, *s;  
    while (p->next != NULL && p->next->data != x)  
        p=p->next;
```

```
    if (p->next != NULL)
```

```
    {  
        s=p->next
```

```
        p->next=s->next;
```

```
        free(s);
```

```
    }
```

```
    else printf("值为x的结点不存在");
```

```
}
```

// 引入s的目的

free(s)

(2) 头插法，插入一个数据为x的结点

```
void insert (LNode *head, int x)
```

```
{  
    LNode *s;
```

```
    s=(LNode *) malloc (sizeof(LNode));
```

```
    s->data=x;
```

```
    s->next=head->next;
```

```
    head->next=s;
```

```
}
```

2. (1) 顶点 v 的出度

```
int outdegree ( MGraph *g, int i)
{
    int count=0; //记录第i顶点出度个数
    for ( int j=0; j < g->vernum; j++)
        if ( g->AdjMatrix[i][j] == 1)
            count++;
    return count;
}
```

(2) 顶点 v 的入度

```
int indegree ( MGraph *g, int i)
{
    int count=0;
    for ( int j=0; j < g->vernum; j++)
        if ( g->AdjMatrix[j][i] == 1)
            count++;
    return count;
}
```

3. 快速排序

```
#define MAXSIZE 20

typedef struct {
    int key;
    InfoType otherinfo;
} RedType;

typedef struct {
    RedType r[MAXSIZE+1];
    int length;
} SqList;

void Quicksort ( SqList &L, int low, int high)
{
    int i;
    if ( low < high ) {
        i = Partition ( L, low, high );
        Quicksort ( L, low, i-1 );
        Quicksort ( L, i+1, high );
    }
}

int Partition ( SqList &L, int low, int high)
{
    L.r[0] = L.r[low]; //将首元素存放在r[0]
    int pivotkey = L.r[low].key;
    while ( low < high )
    {
        while ( low < high && L.r[high].key >= pivotkey )
            --high;
        L.r[low] = L.r[high];
        while ( low < high && L.r[low].key <= pivotkey )
            ++low;
        L.r[high] = L.r[low];
    }
    L.r[low] = L.r[0];
    return low;
}
```